

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
divya@Divyasree:~$ cd week3
divya@Divyasree:~/week3$ nano task1.c
divya@Divyasree:~/week3$ gcc task.c -o task1
cc1: fatal error: task.c: No such file or directory
compilation terminated.
divya@Divyasree:~/week3$ gcc task1.c -o task1
divya@Divyasree:~/week3$ ./task1
myshell$ hello
You entered: hello
myshell$ abc
You entered: abc
myshell$ exit
Shell exited.
divya@Divyasree:~/week3$ |
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. Open()
2. Read()
3. Write()
4. Close()

```
Shell exited.
divya@Divyasree:~/week3$ pwd
/home/divya/week3
divya@Divyasree:~/week3$ nano File1.txt
divya@Divyasree:~/week3$ cat File1.txt
Hello, this is File1.
This content will be copied to File2.
divya@Divyasree:~/week3$ nano task2.c
divya@Divyasree:~/week3$ gcc task2.c -o task2
divya@Divyasree:~/week3$ ./task2
File copied successfully.
divya@Divyasree:~/week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c
divya@Divyasree:~/week3$ cat File2.txt
Hello, this is File1.
This content will be copied to File2.
divya@Divyasree:~/week3$ cd ~/week3
divya@Divyasree:~/week3$ nano File1.txt
divya@Divyasree:~/week3$ cat File1.txt
Hello, this is File1.
This content will be copied to File2.
hi this is divya
divya@Divyasree:~/week3$ nano task2.c
divya@Divyasree:~/week3$ cat task2.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
```

```

    char buffer[100];
    int n;

    fd1 = open("File1.txt", O_RDONLY);

    fd2 = open("File2.txt", O_CREAT | O_WRONLY, 0644);

    n = read(fd1, buffer, 100);

    write(fd2, buffer, n);

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

    return 0;
}
divya@Divyasree:~/week3$ gcc task2.c -o task2
divya@Divyasree:~/week3$ ./task2
File copied successfully.
divya@Divyasree:~/week3$ cat File2.txt
Hello, this is File1.
This content will be copied to File2.
hi this is divya
divya@Divyasree:~/week3$ ./task2
File copied successfully.
divya@Divyasree:~/week3$ cat File2.txt
Hello, this is File1.
This content will be copied to File2.
hi this is divya
divya@Divyasree:~/week3$ |

```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```

divya@Divyasree:~/week3$ nano task3.c
divya@Divyasree:~/week3$ cat task3.c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

int main()
{
    char buffer[100];
    char ch;
    int index = 0;

    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    write(STDOUT_FILENO, "mysh$ ", 6);

    while(1)
    {
        read(STDIN_FILENO, &ch, 1);

        if(ch == '\n' || ch == '\r')
        {
            buffer[index] = '\0';

```

```

        write(STDOUT_FILENO, "\n", 1);

        break;
    }

    else if(ch == 127 || ch == 8)
    {
        if(index > 0)
        {
            index--;

            write(STDOUT_FILENO, "\b \b", 3);

        }
    }

    else
    {
        if(index < 99)
        {
            buffer[index] = ch;
            index++;

            write(STDOUT_FILENO, &ch, 1);

        }
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

printf("Command: %s\n", buffer);

return 0;

```

```

    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

printf("Command: %s\n", buffer);

return 0;
}
divya@Divyasree:~/week3$ gcc task3.c -o task3
divya@Divyasree:~/week3$ ./task3
mysh$ hello
Command: hello
divya@Divyasree:~/week3$ ./task3
mysh$ helll
Command: helll
divya@Divyasree:~/week3$ ./task3
mysh$ hell
Command: hell
divya@Divyasree:~/week3$ ./task3
mysh$ hello123
Command: hello123
divya@Divyasree:~/week3$ |

```

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
divya@Divyasree:~/week3$ nano task4.c
divya@Divyasree:~/week3$ gcc task4.c -o task4
divya@Divyasree:~/week3$ ./task4
mysh$ hello
Command: hello
mysh$ world
Command: world
mysh$ test123
Command: test123
mysh$
divya@Divyasree:~/week3$ |
```

```
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define MAX 100
#define HISTORY_SIZE 10

int main()
{
    char buffer[MAX];
    char history[HISTORY_SIZE][MAX];
    char ch;
    int index = 0;
    int history_count = 0;
    int history_pos = 0;

    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while(1)
    {
        write(STDOUT_FILENO, "mysh$ ", 6);
        write(STDOUT_FILENO, "mysh$ ", 6);

        index = 0;
        history_pos = history_count;

        while(1)
        {
            read(STDIN_FILENO, &ch, 1);

            if(ch == '\n' || ch == '\x1b')
            {
                buffer[index] = '\0';
                write(STDOUT_FILENO, "\n", 1);

                if(strcmp(buffer, "exit") == 0)
                {
                    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
                    return 0;
                }

                if(index > 0)
                {
                    strcpy(history[history_count % HISTORY_SIZE], buffer);

                    if(history_count < HISTORY_SIZE)
                        history_count++;

                    history_pos = history_count;
                }
            }
        }
    }
}
```

```
        printf("Command: %s\n", buffer);
        break;
    }

    else if(ch == 127 || ch == 8)
    {
        if(index > 0)
        {
            index--;
            write(STDOUT_FILENO, "\b\b", 3);
        }
    }

    else if(ch == 27)
    {
        char seq[2];

        read(STDIN_FILENO, &seq[0], 1);
        read(STDIN_FILENO, &seq[1], 1);

        if(seq[0] == '[' && seq[1] == 'A')
        {
            if(history_count > 0 && history_pos > 0)
            {
                history_pos--;

                strcpy(buffer, history[history_pos]);
            }
        }
    }
}
```

^G Help **^O** Write Out **^W** Where Is **^K** Cut **^T** Execute **^C** Location **M-U** Undo
^X Exit **^R** Read File **^N** Replace **^U** Paste **^J** Justify **^/_** Go To Line **M-E** Redo

```
        index = strlen(buffer);

        write(STDOUT_FILENO, "\r\033[K", 4);
        write(STDOUT_FILENO, "mysh$ ", 6);
        write(STDOUT_FILENO, buffer, index);
    }
}

else if(seq[0] == '[' && seq[1] == 'B')
{
    if(history_pos < history_count - 1)
    {
        history_pos++;

        strcpy(buffer, history[history_pos]);

        index = strlen(buffer);

        write(STDOUT_FILENO, "\r\033[K", 4);
        write(STDOUT_FILENO, "mysh$ ", 6);
        write(STDOUT_FILENO, buffer, index);
    }
    else
    {
        history_pos = history_count;
        index = 0;
        buffer[0] = '\0';
    }
}
```

```

        }
        history_pos = history_count;
        index = 0;
        buffer[0] = '\0';

        write(STDOUT_FILENO, "\r\033[K", 4);
        write(STDOUT_FILENO, "mysh$ ", 6);
    }
}

else
{
    if(index < MAX - 1)
    {
        buffer[index] = ch;
        index++;

        write(STDOUT_FILENO, &ch, 1);
    }
}
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}

```

Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```

divya@Divyasree:~/week3$ nano task5.c
divya@Divyasree:~/week3$ gcc task5.c -o task5
divya@Divyasree:~/week3$ ./task5
Enter some text: Hello this is Divya
You entered: Hello this is Divya

Updated string: Hello this is Divya
- Memory Resized
Linked List:
Node 1 -> Node 2 -> Node 3 -> Node 4 -> Node 5 -> NULL
Memory released successfully.
divya@Divyasree:~/week3$ valgrind --version
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
divya@Divyasree:~/week3$ sudo apt update
[sudo] password for divya:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
75 packages can be upgraded. Run 'apt list --upgradable' to see them.
divya@Divyasree:~/week3$ valgrind --version
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
divya@Divyasree:~/week3$ ^C
divya@Divyasree:~/week3$ sudo snap install valgrind

```

```

divya@Divyasree:~/week3$ sudo snap install valgrind
error: This revision of snap "valgrind" was published using
       classic confinement and thus may perform arbitrary system
       changes outside of the security sandbox that snaps are
       usually confined to, which may put your system at risk.

       If you understand and want to proceed repeat the command
       including --classic.
divya@Divyasree:~/week3$ sudo apt install valgrind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common
  libdebuginfod1t64 libipt2 libsource-highlight-common
  libsource-highlight4t64
Suggested packages:
  gdb-doc gdbserver valgrind-dbg valgrind-mpi kcachegrind
  alioop valkyrie
The following NEW packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common
  libdebuginfod1t64 libipt2 libsource-highlight-common
  libsource-highlight4t64 valgrind
0 upgraded, 9 newly installed, 0 to remove and 75 not upgraded.
Need to get 27.0 MB of archives.
After this operation, 103 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod-common all 0.190-1.1ubuntu0.1 [14.6 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 libbabeltrace1 amd64 1.5.11-3build3 [164 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod1t64 amd64 0.190-1.1ubuntu0.1 [17.1 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 libipt2 amd64 2.0.6-1build1 [45.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/main amd64 libsource-highlight-common all 3.1.9-4.3build1 [64.2 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 libsource-highlight4t64 amd64 3.1.9-4.3build1 [258 kB]
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
divya@Divyasree:~/week3$ valgrind --version
valgrind-3.22.0
divya@Divyasree:~/week3$ valgrind --leak-check=full ./task5
==3391== Memcheck, a memory error detector
==3391== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==3391== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==3391== Command: ./task5
Enter some text: Hello this is Divya
You entered: Hello this is Divya

Updated string: Hello this is Divya
- Memory Resized
Linked List:
Node 1 -> Node 2 -> Node 3 -> Node 4 -> Node 5 -> NULL
Memory released successfully.
==3391==
==3391== HEAP SUMMARY:
==3391==    in use at exit: 0 bytes in 0 blocks
==3391==   total heap usage: 6 allocs, 6 frees, 1,344 bytes allocated
==3391==
==3391== All heap blocks were freed -- no leaks are possible
==3391==
==3391== For lists of detected and suppressed errors, rerun with: -s
==3391== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
divya@Divyasree:~/week3$

```